Colecțiile din .NET și LINQ — Teorie

Ștefan · Partea 2 · punte spre `List`, `Dictionary`, `HashSet`, `Queue`, `Stack`, egalitatea obiectelor, LINQ

> **Exemplul din repo:** `etapa-2-colectii-csharp/ExempluAeroport.cs` (în proiectul `Colections`, namespace `Colections.Etapa2`) are 12 secțiuni în ordinea teoriei de mai jos, pe date de aeroport: zboruri, coada la check-in, bagajele din cală. Îl chemi din `Main` cu `Colections.Etapa2.ExempluAeroport.Ruleaza();`, citești ieșirea, apoi codul secțiune cu secțiune. Ieșirile din fișa asta vin din rularea lui pe .NET 8.

0. Mediul: același proiect, dar C# 7.3 la olimpiadă

Lucrezi în proiectul `Colections`, pe **.NET 8**. La olimpiadă ai **.NET Framework 4.x**, unde limbajul implicit e **C# 7.3**. Compilatorul tău de acum **nu te oprește** dacă scrii sintaxă nouă: tu te ții de tabel, în exemple și în exerciții.

Nu merge în C# 7.3	Scrii așa
`List<int> l = new();`	`List<int> l = new List<int>();`
`using var w = ...;`	`using (var w = ...) { ... }`
`record`, `is not null`, `x switch { ... }`	clasă obișnuită, `!= null`, `switch` clasic
`string? s`, `object? o`	`string s`, `object o`

Nici **metodele** din .NET nu sunt aceleași. Pe Framework 4.x lipsesc (verificat prin compilare cu bibliotecile lui 4.7.2):

Nu există în 4.x	Scrii așa
`HashCode.Combine(a, b)`	`a.GetHashCode() * 31 + b.GetHashCode()`
`d.TryAdd(k, v)`	`if (!d.ContainsKey(k)) { d.Add(k, v); }`
`s.TryPop(out x)`, `q.TryDequeue(out x)`	`if (s.Count > 0) { x = s.Pop(); }`
`d.EnsureCapacity(n)`, `DistinctBy`, `MinBy`	nu le folosești

Pentru LINQ ai nevoie de `using System.Linq;` sus în fișier. Proiectul tău îl are global, dar la olimpiadă nu: îl scrii. La fel `using System;` și `using System.Collections.Generic;`.

1. Ce ai scris tu ↔ ce are .NET

În partea 1 ți-ai construit colecțiile. .NET le are gata făcute, cu aceleași idei dedesubt:

Ale tale	În .NET	Ce diferă
`ListaTablou<T>`	`List<T>`	aceleași nume: `Count`, `Capacity`, `Add`, `Insert`, `RemoveAt`, `Remove`, `IndexOf`, `Contains`, `Clear`, `ToArray`, `this[i]`; `Capacity` pornește de la 0, nu de la 4
`ListaInlantuita<T>`	`LinkedList<T>`	nu are `this[i]`, `Insert(i, x)`, `RemoveAt(i)`; are `AddFirst`, `AddLast`, `Remove(x)`, `First`, `Last`
`StivaTablou<T>`	`Stack<T>`	`Push`, `Pop`, `Peek`; `foreach` pornește de la vârf
`CoadaCirculara<T>`	`Queue<T>`	`Enqueue`, `Dequeue`, `Peek`
`DictionarHash<K, V>`	`Dictionary<K, V>`	`TryGetValue` în loc de `TryToTake`; nu arată câte găleți are
`MultimeHash<T>`	`HashSet<T>`	`UnionWith`, `IntersectWith`, `ExceptWith` **modifică** mulțimea; `Reuniune` / `Intersectie` ale tale întorc una nouă

Mecanismele pe care le-ai construit se regăsesc, dar cu câteva diferențe care contează. Toate sunt rulate pe .NET 8 (`ExempluAeroport`):

Mecanismul tău	În .NET

Dublarea tabloului, `Capacity`	`List<T>` o face la fel: 0 → 4 → 8 → 16 (ieșirea de mai jos)
`versiune` → `InvalidOperationException` în `foreach`	`List<T>`: orice `Add`, `Remove`, `this[i] = ...` în timpul `foreach` aruncă. `Dictionary`: o cheie nouă aruncă, dar **suprascrierea unei chei existente și `Remove` nu aruncă** (pe .NET 8)
`TryToTake` (în codul tău scoate cheia)	`TryGetValue` **doar citește**: `Count` și cheia rămân
`Add` pe cheie existentă	`Add` aruncă `ArgumentException`; `d[k] = v` înlocuiește sau adaugă; `d[k]` pe cheie lipsă aruncă `KeyNotFoundException`
Adăugare în mulțime	`HashSet.Add(x)` întoarce `bool`: `true` dacă x e nou
`Equals` + `GetHashCode` în `Pozitie`	aceeași regulă (§8). `Equals(null)` întoarce `false`, nu aruncă. `HashCode.Combine` există doar în .NET modern (§0)
Excepțiile din Pas 6	aceleași tipuri. Mesajele diferă puțin: `List.Insert` și `this[i]` au mesaje diferite, iar `LinkedList` are alt mesaj la modificare în `foreach`

```
Count=0 Capacity=0
dupa Add #1: Capacity=4
dupa Add #5: Capacity=8
dupa Add #9: Capacity=16
this[20]: Index was out of range
Insert(20, 1): Index must be within the bounds of the List
dupa TryGetValue: Count=6 ContainsKey=True
Add pe cheie existenta -> ArgumentException
indexerul inlocuieste: Viena, Count=6
foreach + mici["c"] = 3 -> InvalidOperationException
foreach + mici["a"] = 9 -> ok
foreach + Remove -> ok
True True False True True False
```

Nu te baza pe ce nu aruncă. Că `Remove` în `foreach` merge pe un `Dictionary` e comportamentul lui .NET 8. Pe Framework 4.x nu l-am rulat aici, așa că presupui că aruncă. Regula sigură rămâne cea de la §4: modifici o colecție doar în afara unui `foreach` pe ea.

2. Tablou sau `List<T>`

	Tablou `int[]`	`List<int>`
Mărime	fixă, dată la creare	crește singură la `Add`
Câte elemente	`v.Length`	`l.Count`
Când îl alegi	știi dinainte câte sunt (matrice, 7 zile)	aproape mereu altfel

Pe dedesubt, `List<T>` ține tot un tablou, ca `ListaTablou` a ta. Când se umple, alocă unul de **două ori mai mare** și copiază tot în el. De aceea `Add` e rapid în medie, deși uneori copiază.

3. Operațiile din `List<T>`

Metoda	Ce face	Atenție
`Add(x)` / `Insert(i, x)`	adaugă la coadă / pe poziția `i`	`Insert` mută tot ce e după `i`
`Remove(x)`	scoate **prima** apariție a lui `x`	întoarce `bool`: a găsit-o sau nu
`RemoveAt(i)`	scoate elementul de pe poziția `i`	toate cele de după coboară cu o poziție
`Contains(x)` / `IndexOf(x)`	caută `x`	parcurge toată lista; compară cu `Equals` (§8)

`Sort()`	sortează pe loc	pentru obiecte îi dai regula: `l.Sort((a, b) => a.X.CompareTo(b.X))`

4. Modifici lista în timp ce o parcurgi

Trei situații, toate verificate prin rulare:

- **`foreach` + `Remove`/`Add`** → `InvalidOperationException: Collection was modified`. E fail-fast-ul din Pas 6: `foreach` verifică la fiecare pas că lista nu s-a schimbat sub el.
- **`for` crescător + `RemoveAt(i)`** → sare peste elemente. Pe `{ -1, -2, 3 }`, „șterge negativele” lasă `-2 3`. După ce ștergi poziția 0, `-2` coboară pe poziția 0, dar `i` trece la 1.
- **`for` crescător + `Insert(i + 1, ...)`** → elementul inserat ajunge exact pe poziția pe care o verifici la pasul următor. Dacă și el îndeplinește condiția, inserezi din nou, la nesfârșit.

Regula: când ștergi sau inserezi în timp ce parcurgi, parcurgi **de la coadă spre cap** (`for (int i = l.Count - 1; i >= 0; i--)`). Ce schimbi rămâne în urma ta, unde nu te mai uiți.

5. `Dictionary<TKey, TValue>`

O colecție de perechi **cheie → valoare**, cu cheile unice. Găsește valoarea după cheie **direct**, fără să parcurgă tot, oricât de mare ar fi: `DictionarHash` al tău, cu găleți.

```
Dictionary<string, string> capitale = new Dictionary<string, string>();
capitale.Add("Franta", "Paris");
capitale["Italia"] = "Roma";
```

Operația	Dacă cheia există	Dacă cheia NU există
`d.Add(k, v)`	❌ `ArgumentException`	adaugă
`d[k] = v`	înlocuiește valoarea	adaugă
`d[k]` (citire)	valoarea	❌ `KeyNotFoundException`
`d.TryGetValue(k, out v)`	`true` + valoarea în `v`; dicționarul nu se schimbă	`false`, iar `v` primește valoarea implicită (`0`, `null`)
`d.ContainsKey(k)`	`true`	`false`

Parcurgerea: `foreach (KeyValuePair<string, string> p in capitale)`, cu `p.Key` și `p.Value`. Mai ai `capitale.Keys` și `capitale.Values`.

> **Ordinea nu e garantată.** Documentația nu promite nicio ordine la parcurgere. Dacă trebuie să afișezi într-o ordine anume, pui cheile într-o listă și o sortezi: `List<string> chei = new List<string>(d.Keys); chei.Sort();`

Tiparul numărătorului, cel mai des folosit:

```
int deja;
d.TryGetValue(cheie, out deja);
d[cheie] = deja + 1;
```

Dacă cheia nu există, `deja` rămâne `0`, deci prima apariție scrie `1`.

6. `HashSet<T>`

O mulțime: fiecare element apare **o singură dată**, fără ordine și fără poziții. Ca `MultimeHash` a ta, peste un dicționar.

- `s.Add(x)` întoarce `true` dacă `x` e nou și `false` dacă exista deja. Un singur apel înseamnă și „verifică”, și „adaugă”.
- `s.Contains(x)` e **direct**, ca la dicționar. `List.Contains` parcurge tot.
- Operații de mulțime: `UnionWith`, `IntersectWith`, `ExceptWith`. Ele **modifică** mulțimea pe care le chemi, nu întorc una nouă. Dacă vrei să păstrezi originalul, faci întâi o copie: `new HashSet<string>(a)`.

7. `Queue<T>` și `Stack<T>`

	`Queue<T>` : coadă	`Stack<T>` : stivă

Regula	primul intrat, primul ieșit	ultimul intrat, primul ieșit
Pune	`Enqueue(x)`	`Push(x)`
Scoate	`Dequeue()`	`Pop()`
Se uită, fără să scoată	`Peek()`	`Peek()`
Exemple	coada la ghișeu, ordinea de procesare	Undo, paranteze, drumul înapoi

`Dequeue` / `Pop` / `Peek` pe o colecție goală aruncă `InvalidOperationException` (`Queue empty.` / `Stack empty.`, ca în Pas 6). Verifici `Count > 0` înainte.

8. Când sunt două obiecte „la fel”?

Pentru o clasă scrisă de tine, implicit, două obiecte sunt egale doar dacă sunt **același obiect** (aceeași referință), exact ce verifica `ReferenceEquals` la Singleton. Două obiecte cu aceleași câmpuri, create cu două `new`, sunt diferite. Deci `List.Contains` nu le găsește, iar `HashSet` le ține pe amândouă. Același lucru l-ai văzut la `Pozitie`, în Pas 5c.

Ca să le spui ce înseamnă „la fel”, suprascrii două metode, **întotdeauna împreună**:

```
public override bool Equals(object obj)
{
    Bilet altul = obj as Bilet;
    if (altul == null)
    {
        return false;
    }
    return CodZbor == altul.CodZbor && Loc == altul.Loc;
}

public override int GetHashCode()
{
    return CodZbor.GetHashCode() * 31 + Loc.GetHashCode();
}
```

De ce amândouă: `HashSet` și `Dictionary` se uită **întâi** la `GetHashCode`, ca să știe în ce „sertar” să caute, și abia apoi compară cu `Equals`. Regula e: **obiecte egale ⇒ același hash**. Dacă suprascrii doar `Equals`, două bilete egale ajung în sertare diferite și nu se întâlnesc niciodată. `List.Contains` folosește doar `Equals`.

`Equals` primește orice `object`, deci trebuie să întoarcă `false` pentru `null` și pentru alt tip, fără să arunce: de asta `as` și verificarea de `null` vin primele.

Hash-ul poate fi orice formulă, cât timp obiectele egale dau același număr. Cu `+` (`X + Y`), punctele (1, 2) și (2, 1) dau același hash; cu `* 31` ordinea contează și dau numere diferite. La olimpiadă n-ai `HashCode.Combine` (§0), deci folosești formula cu `* 31`.

Legătura cu academy: verificarea de duplicat pe `Guid` nu putea prinde nimic. `Equals` pe `Id` nu spune nimic despre ce înseamnă „aceeași carte”.

9. Ce colecție alegi

Întrebarea ta	Colecția
„Am o înșiruire în ordine, cu poziții”	`List<T>`
„Caut ceva după o cheie (nume, cod)”	`Dictionary<K, V>`
„Am văzut deja asta?” / „fără dubluri”	`HashSet<T>`
„Procesez în ordinea sosirii”	`Queue<T>`
„Mă întorc la ultimul lucru”	`Stack<T>`
„Știu sigur câte sunt, nu se schimbă”	tablou

10. LINQ: interogări pe colecții

LINQ e un set de metode care se pot chema pe **orice colecție** (tot ce e `IEnumerable<T>`: liste, tablouri, dicționare, mulțimi). Fiecare primește o **lambda**, o funcție scurtă scrisă pe loc:

```
e => e.Medie >= 9
```

Se citește „pentru un `e`, întoarce `e.Medie >= 9`”. E o metodă fără nume: `e` e parametrul, iar după `=>` vine ce întoarce.

Operatorul	Ce face	Echivalentul cu buclă
`Where(e => cond)`	păstrează doar ce trece de condiție	`foreach` + `if` + `Add`
`Select(e => ...)`	transformă fiecare element	`foreach` + `Add(transformat)`
`OrderBy` / `OrderByDescending`, apoi `ThenBy`	sortează după o cheie, apoi după a doua	`Sort` cu regulă
`Count()`, `Sum`, `Average`, `Min`, `Max`	un singur număr din toată colecția	o variabilă acumulator
`Any(cond)` / `All(cond)`	măcar unul / toți	o variabilă `bool` + `break`
`First(cond)` / `FirstOrDefault(cond)`	primul care trece	`foreach` + `return`
`Distinct`, `Intersect`, `Except`, `Union`	operații de mulțime	`HashSet`
`GroupBy(e => cheie)`	grupuri: fiecare are `Key` și elementele lui	`Dictionary<K, List<T>>`
`ToList()`, `ToDictionary(...)`	pune rezultatul într-o colecție nouă	—

```
List<string> pline = zboruri.Where(z => z.Pasageri > 100)
                            .OrderByDescending(z => z.Pasageri)
                            .Select(z => z.Cod + " " + z.Pasageri)
                            .ToList();
```

Pe zborurile din `ExempluAeroport`, `pline` dă:

```
[W6303 180, W6404 175, RO101 150, LH505 120]
```

> **`First` vs `FirstOrDefault`.** Dacă nu găsește nimic, `First` aruncă `InvalidOperationException`, iar `FirstOrDefault` întoarce `null` (sau `0`). Folosești `First` doar când ești sigur că elementul există. Altfel `FirstOrDefault` + verificare de `null`.

LINQ nu modifică colecția: întoarce mereu una nouă. Pentru „șterge din listă tot ce...” rămâne bucla de la §4.

11. LINQ e amânat

`Where` nu filtrează nimic când îl scrii. Întoarce o **rețetă**, care se execută abia când parcurgi rezultatul (`foreach`, `string.Join`, `Count()`, `ToList()`).

```
List<int> intarzieri = new List<int> { 5, 40, 12 };
IEnumerable<int> mari = intarzieri.Where(m => m > 30);
List<int> mariAcum = intarzieri.Where(m => m > 30).ToList();
intarzieri.Add(90);
```

- `mari` dă `40, 90`: rețeta rulează la afișare, pe lista de atunci;
- `mariAcum` dă `40`: e o copie făcută înainte de `Add`.

Când vrei rezultatul „de acum”, pui `.ToList()` la final.